

NYCU 2025 DCS 數位電路與系統 期末考

考試時間: 2025/05/27 15:30~17:20 請確實填寫姓名、學號與工作站帳號，題目卷會回收

姓名：

學號：

工作站帳號：

1. (10%) (Pipeline and parallel) Given there is a pipelined design with three stages, A, B, and C. The latency for A, B, and C are 8/16/4 cycles, respectively.

- (a) (2%) Plot the pipeline diagram for four consecutive inputs. Indicate the hardware utilization rate for A, B, and C, respectively.

- (b) (3%) How can this design be pipelined using the minimum number of stages while maximizing hardware utilization?

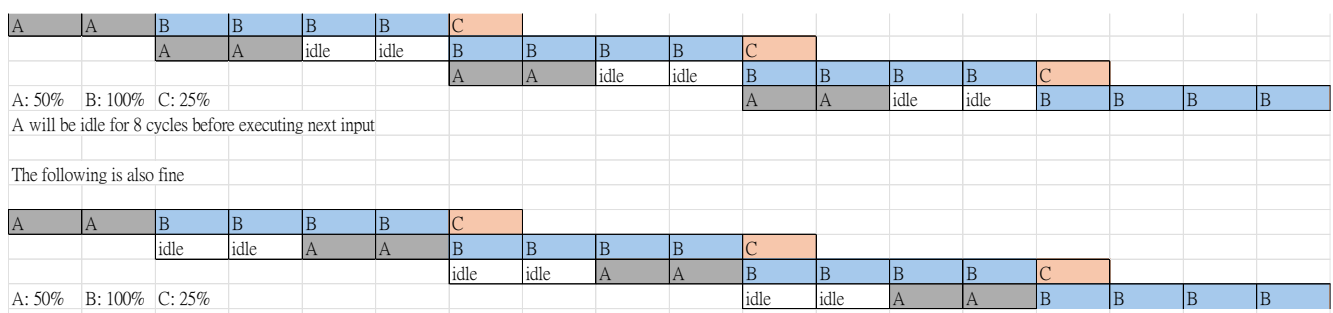
- (c) (2%) Following (a), assume the B stage has variable delay controlled by an additional input: 16 cycles when the additional input is 1, and 4 cycles when it is 0. Plot the pipeline diagram for four consecutive inputs, where the additional input sequence is 1010.

- (d) (3%) How to prevent the design in (c) from stall? Plot your block diagram.

Ans:

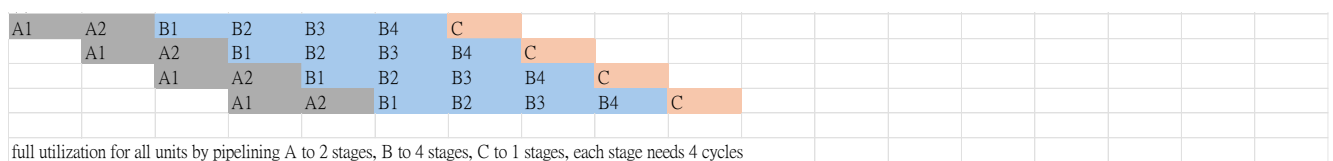
(a) 兩種畫法皆可，pipeline diagram 正確 1 分，utilization rate 正確 1 分

Utilization rate: A=50% B=100% C=25%

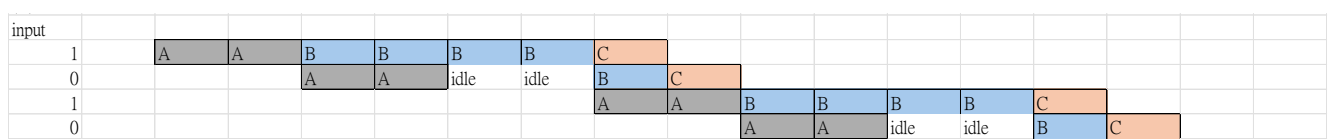


(註:每一格是 4 cycles)

(b) 寫出 A 切 2 stage，B 切 4 stage 得 3 分



(c) 共有 3 個 stage(A,B,C)的訊號，ABC 1 種有錯扣 1 分，扣完為止



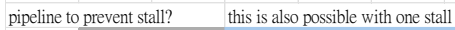
(d) 若提出方法合理得 2 分，電路 block diagram 正確得 1 分

你已經被期末好運貓貓造訪

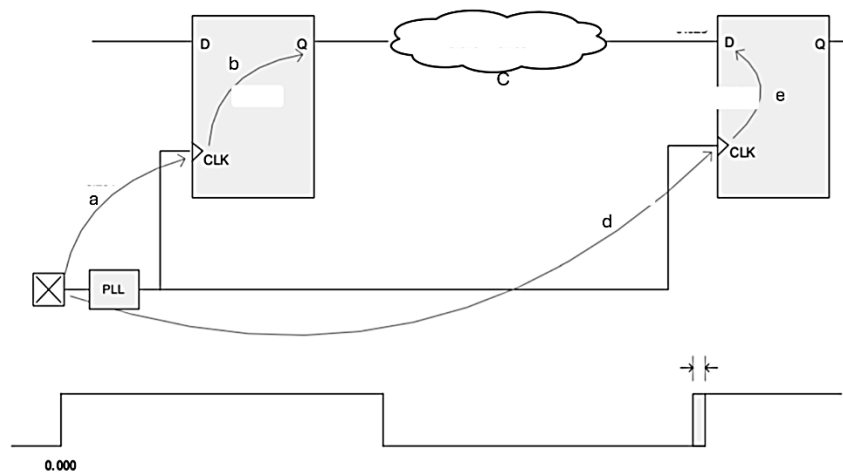


你可以分享這張圖並且打出
「感謝期末好運貓貓」
但分數並不會因此比較高

one approach is to use parallelism (use four copies of B and two copies to A. This makes all stages consume 4 cycles



- | Point | Incr | Path |
|---|-------|--------|
| clock clk (rise edge) | 0.00 | 0.00 |
| clock network delay (propagated) | 0.15 | 0.15 |
| reg_file/data_reg[15]/CK (DFCNQD1BWP7T30P140) | 0.00 | 0.15 r |
| reg_file/data_reg[15]/Q (DFCNQD1BWP7T30P140) | 0.12 | 0.27 r |
| reg_file/data_out[15] (reg_file) | 0.00 | 0.27 r |
| U156/A1 (NAND2D1BWP7T30P140) | 0.00 | 0.27 r |
| U156/Z (NAND2D1BWP7T30P140) | 0.04 | 0.31 f |
| U157/A1 (INVD1BWP7T30P140) | 0.01 | 0.32 f |
| U157/Z (INVD1BWP7T30P140) | 0.02 | 0.34 r |
| alu_out_reg[7]/D (DFCNQD1BWP7T30P140) | 0.00 | 0.34 r |
| data arrival time | | 0.34 |
| clock clk (rise edge) | 1.00 | 1.00 |
| clock network delay (propagated) | 0.18 | 1.18 |
| clock uncertainty | -0.05 | 1.13 |
| alu_out_reg[7]/CK (DFCNQD1BWP7T30P140) | 0.00 | 1.13 r |
| library setup time | -0.04 | 1.09 |
| data required time | | 1.09 |
| data required time | | 1.09 |
| data arrival time | | -0.34 |
| slack (MET) | | 0.75 |



a: 0.15, b: 0.12, c: 0.07, d: 0.18, e: 0.04 (一個兩分)

- 2

	$Q_1 Q_2 Q_3$	$Q_1^+ Q_2^+ Q_3^+$		Z	
		$X=0$	$X=1$	$X=0$	$X=1$
A	000	100	101	1	0
B	100	111	110	1	0
C	101	110	110	0	1
D	111	011	011	0	1
E	110	011	010	1	0
H	011	000	000	0	1
M	010	000	x x x	1	x
-	001	x x x	x x x	x	x

- (a) (6%) Microcode is a commonly used method to implement FSM with ROM table. List the ROM contents for the following FSM table. You shall list the address and data of the ROM contents.
- (b) (4%) The memory size could reach a large one if the state and input is increasing. Could you propose two methods to decrease the memory size?

Ans:

(a)

Address {Q1, Q2, Q3, X}	Data {Q1+, Q2+, Q3+, Z}
0000	1001
0001	1010
0010	xxxx
0011	xxxx
0100	0001
0101	xxxx
0110	0000
0111	0001
1000	1111
1001	1100
1010	1100
1011	1101
1100	0111
1101	0100
1110	0110
1111	0111

需完整列出 16 個 address 與 data 的組合(排列方式可以不同)，其中三個資料為 x 的欄位可以不用寫，以上表格每錯一組扣一分，扣到該題零分為止。另外，若 Address 或 Data 未標示 bit 分配方式(ex. {state, X})扣一分。

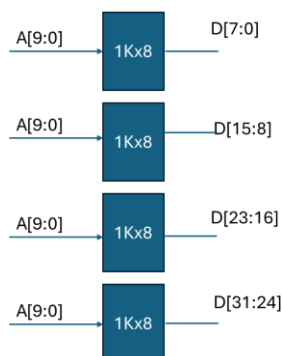
- (b) Split states and output. Furthermore, use instruction and program counter to replace states.

每個有效方法兩分

4. (10%) (Memory usage) Assume you have a basic 1Kx8bits SRAM. For the following problem, plot the block diagram and specify your address and data bits clearly.

(a) (2%) Build a 1Kx32bits SRAM with the basic SRAM block.

(a) All 1Kx8 use the same address A[9:0], concatenate their output to be 32 bits

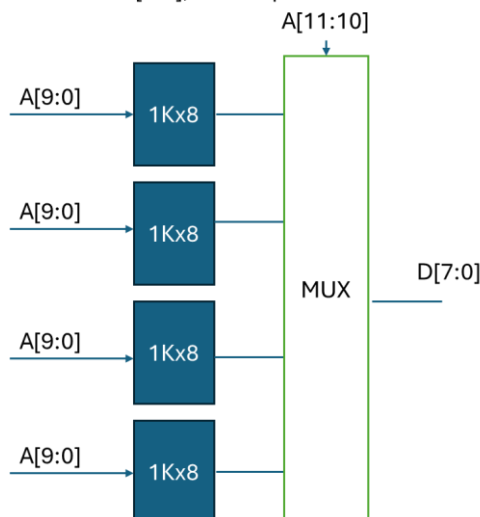


得分	標準
1	畫出四個 SRAM
2	清楚標明所有接線

補充說明：應清楚標明將輸入與輸出資料每 8 bit 一組分別放在四個 SRAM 才能算做清楚標明接線。

(b) (3%) Build a 4Kx8bits SRAM with the basic SRAM block.

(b) All 1Kx8 use the same address A[9:0], The output is selected from the one of banks



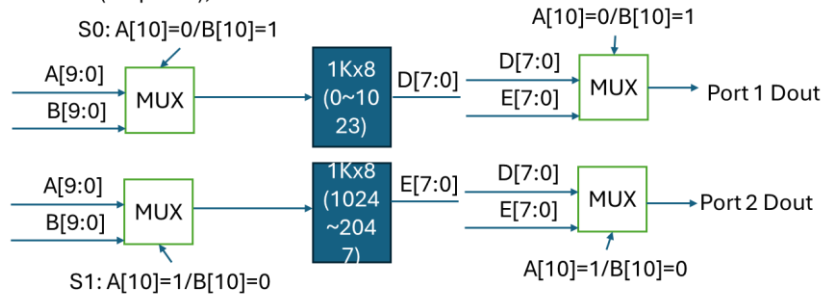
得分	標準
1	畫出四個 SRAM
2	在輸出部分畫出 MUX
3	清楚標明所有接線

補充說明：若是沒有提及使用 2 bit 做選擇會視作沒有清楚標明接線(拿不到 3 分)。另外其實 data in 的部分也應該要使用 A[11:10]去選擇是否輸入，但是助教確認上課簡報中沒有相關範例(都只畫

了 data out)，因此不會特別檢查此部分。

- (c) (5%) The basic SRAM block only allow one read or one write. Build a memory system (2Kx8bits) with the basic SRAM block that allows two ports access (one read and one write per cycle or two read per cycle or two write per cycle. Assume the two ports access different addresses. There are two address spaces divided evenly, and the two ports won't access the same address space at the same time.) Hint. Use two memory blocks. Add additional logic if necessary. Explain how it works.

(c)SRAM 1 (associated with port 1), address input: select one of two port address lines
Similar for SRAM 2 (for port 2), other control like CE or write or read can be omitted



SRAM1/2 Data Input (Din0-Din7):

- MUX inputs: Din0_0-Din0_7 (Port 0) or Din1_0-Din1_7 (Port 1).
- Select: Same as address (S0/S1).

Block diagram 部分佔 4 分

得分	標準
1	畫出二個 SRAM
2	畫出兩個 MUX(在輸入或輸出)
3	畫出四個 MUX(輸入與輸出各兩個，或 address/data in 分開算就是六個)
4	清楚標明所有接線

說明部分佔 1 分。應說明使用 MUX 選擇兩個 address/輸入，以及使用 MUX 選擇兩個輸出。

5. (10%) Metastability and Synchronizer.

- (a) (3%) Metastability is a critical issue in clock domain crossing. What does metastability mean?

metastability 是數位電路中一種不穩定的輸出狀態，當 flip-flop 在有限的時間內無法穩定判斷輸入訊號的高低電平時，就會進入介於高、低電平之間的不確定狀態。這種情況常發生在 clock domain crossing (CDC)，但不僅限於 CDC，只要有訊號在 flip-flop 的 setup time 與 hold time 內改變，都可能導致 metastability。metastability 會造成後續邏輯異常，需透過設計手法降低其發生機率。

補充說明：由於許多人僅回答 metastability 的成因而非定義，若是說明充分足以讓助教相信你知道 metastability 的相關知識，助教也會斟酌給分。

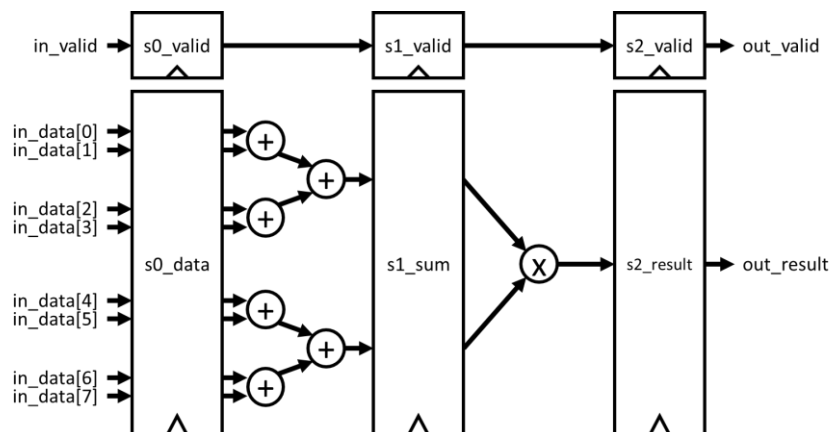
- (b) (3%) How can using two flip-flops prevent metastability during clock domain crossing?

透過串接兩個 flip-flops 作為同步器，第一個 flip-flop 接收另一個 clock domain 的輸入信號，若發生 metastability，它會在下一個 clock cycle 內嘗試穩定輸出；第二個 flip-flop 則再一次對信號取樣，給予足夠的時間讓訊號恢復穩定狀態。這樣設計大幅降低 metastability 傳遞到後續邏輯的機率。

- (c) (4%) Why is Gray code a preferred method for transmitting multi-bit signals across clock domains?

Gray code 的特性是在相鄰的數值間只有一位元會改變(2)，這避免了多位元同時改變造成的瞬間不一致狀態(2)(需要說明訊號不同步抵達的問題)，減少因訊號轉換導致的錯誤或 metastability 發生，提高跨 clock domain 多位元信號傳輸的穩定性和可靠性。

6. (12%) Jerry designed a SystemVerilog circuit based on the block diagram below. The circuit takes eight 8-bit inputs, splits them into two groups of four, sums each group, and multiplies the two sums to produce the final output. He synthesized the design with a 4 ns clock, and the corresponding timing report is provided. Answer the following questions based on the diagram and report.



Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
s1_sum_reg_0__1_/CK (DFFRHQX4)	0.00	0.00 r
s1_sum_reg_0__1_/Q (DFFRHQX4)	0.31	0.31 r
U171/Y (BUFX8)	0.16	0.48 r
U270/Y (XNOR2XL)	0.40	0.88 f
U202/Y (OAI22XL)	0.23	1.11 r
U528/CO (ADDFX1)	0.68	1.79 r
U217/S (ADDFHX1)	0.42	2.21 f
U529/Y (NOR2X1)	0.15	2.36 r
U530/Y (INVX2)	0.07	2.43 f
U543/Y (AOI21X1)	0.15	2.58 r
U544/Y (AOI21X1)	0.15	2.73 f
U546/Y (AOI21X2)	0.24	2.97 r
U554/Y (AOI21X2)	0.17	3.14 f
U573/Y (AOI21XL)	0.22	3.36 r
U578/Y (XOR2X1)	0.25	3.61 f
s2_result_reg_16_/D (DFFRHQXL)	0.00	3.61 f
data arrival time		3.61
clock clk (rise edge)	4.00	4.00
clock network delay (ideal)	0.00	4.00
clock uncertainty	-0.10	3.90
s2_result_reg_16_/CK (DFFRHQXL)	0.00	3.90 r
library setup time	-0.28	3.62
data required time		3.62
data required time		3.62
data arrival time		-3.61
slack (MET)		0.01

- (a) (3%) The timing report shows a slack of 0.01 ns. Does this mean the minimum clock period is 3.99 ns?

Explain.

不能(1)。合成時若使用較小的 cycle time，可能也能夠使用面積較大的硬體(2)(至少要回答出使用不同硬體或架構合成)成功合成。

補充說明：不接受以可能存在小數點後二位以上為理由。實數本就是連續的，無論顯示幾位都可以說他精確度不足，且 slack 仍大於 0，幾乎可以肯定還有優化空間，因此不接受精確度這樣鑽牛角尖的回答。

- (b) (3%) Jerry considers inserting a pipeline stage into the adder tree that connects stage 0 and stage 1 to

reduce the minimum clock period. Would this be effective? **Why or why not?**

不能(1)。 Critical path 位在 s1 到 s2 之間(2)而非 s0 到 s1 之間。因此縮減 s0 到 s1 之間的 path 並不能減少最低 cycle time。

- (c) (3%) Jerry wrote a test task shown below to verify the reset signal. Are there any issues with it? If so, **describe when it may fail and provide a corrected version.**

```
task check_reset; begin
    force clk = 0;
    rst_n = 'b0;
    #(20) if(out_valid != 'b0 || out_result != 'b0) reset_fail;
    rst_n = 'b1;
    #(10); release clk;
end endtask
```

應該使用!==(1)而非!=來檢查訊號(2)。 !=無法檢測 unknown 的訊號(2)，因此當訊號為 unknown 時就會無法偵測出錯誤。

補充說明：若提出在 synchronous reset 時不應使用 force clk / release clk 也給過。確實不應該先入為主地認為設計是 asynchronous reset，因此特別開放此答案。(需要有明確提及 synchronous reset 才給過)

另外有許多同學回答多寫/少寫分號、不應該停住 clock、rst_n 不應一開始就為 0 等等，實際上都不會造成影響，因此不給過。

- (d) (3%) Jerry wants to add a new 2-bit input signal in_mode, which specifies the offset for shifting in_data. For example, if in_mode = 2, in_data is circularly shifted by 2 positions. That is:

$$\text{out_result} = (\text{in_data}[6] + \text{in_data}[7] + \text{in_data}[0] + \text{in_data}[1]) * (\text{in_data}[2] + \text{in_data}[3] + \text{in_data}[4] + \text{in_data}[5])$$

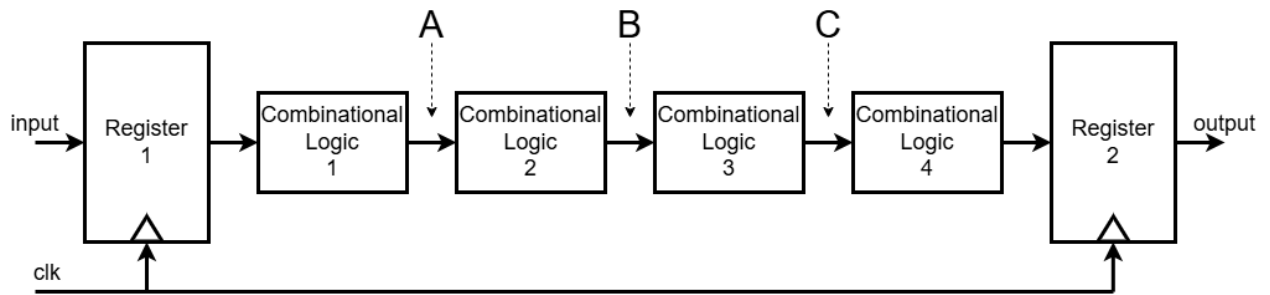
To implement this, both Jerry and his friend, Tom, modified the signal s1_sum_ns, the D input of flip-flop s1_sum. Based on their implementations, which design results in a smaller area? **Explain your reasoning.**

Tom's design:	Jerry's design:
<pre>logic [0:7][7:0] select; always_comb begin case(s0_mode) 2'b00: select = s0_data; 2'b01: select = {s0_data[7], s0_data[0:6]}; 2'b10: select = {s0_data[6:7], s0_data[0:5]}; 2'b11: select = {s0_data[5:7], s0_data[0:4]}; default: select = 'bx; endcase s1_sum_ns[0] = select[0] + select[1] + select[2] + select[3]; s1_sum_ns[1] = select[4] + select[5] + select[6] + select[7]; end</pre>	<pre>always_comb begin case(s0_mode) 2'b00: begin s1_sum_ns[0] = s0_data[0] + s0_data[1] + s0_data[2] + s0_data[3]; s1_sum_ns[1] = s0_data[4] + s0_data[5] + s0_data[6] + s0_data[7]; end 2'b01: begin s1_sum_ns[0] = s0_data[7] + s0_data[0] + s0_data[1] + s0_data[2]; s1_sum_ns[1] = s0_data[3] + s0_data[4] + s0_data[5] + s0_data[6]; end 2'b10: begin s1_sum_ns[0] = s0_data[6] + s0_data[7] + s0_data[0] + s0_data[1]; s1_sum_ns[1] = s0_data[2] + s0_data[3] + s0_data[4] + s0_data[5]; end 2'b11: begin s1_sum_ns[0] = s0_data[5] + s0_data[6] + s0_data[7] + s0_data[0]; s1_sum_ns[1] = s0_data[1] + s0_data[2] + s0_data[3] + s0_data[4]; end default: s1_sum_ns = 'bx; endcase end</pre>

Tom 的設計面積較小(1)。 Jerry 的設計會在四種 mode 開四套硬體，而 Tom 則透過提前選擇資料能夠保持只有一套硬體(2)，因此能夠有較小的面積。

參考面積：Tom(25180.848271), Jerry(32941.339501)

7. (12%) Pipeline.



In the circuit diagram above, Combinational Logic 1~4 are all combinational circuit blocks, and Register 1 and 2 are positive-edge-triggered D flip-flops. The delays of Combinational Logic 1~4 are 3ns, 5ns, 7ns, and 1ns respectively. Assume that other delays are negligible, and **the register setup time is 1ns**, hold time is 0.

- (a) (2%) If you want to insert a pipeline register at one of the locations A, B, or C, which location would be the better choice? Please explain why.

(B) , cycle time 越小 , throughput 可以越大

如果切在 A , 最低 cycle time 為 $5+7+1+1=14\text{ns}$

如果切在 B , 最低 cycle time 為 $3+5+1=9\text{ns}$ 或 $7+1+1=9\text{ns}$

如果切在 C , 最低 cycle time 為 $3+5+7+1=16\text{ns}$

評分標準: 寫對 B 得 2 分

- (b) (2%) Based on the pipeline you made in part (a), what is the shortest possible clock cycle time that will not cause a setup time violation? Please explain why.

切在 B , 最低 cycle time 為 9ns

評分標準: Cycle time 正確得 2 分

- (c) (2%) Calculate the throughput before and after pipelining, based on your choice in part (a). Assume only one output per cycle, and the cycle time is the shortest possible without causing a timing violation.

Before pipeline cycle time: $3+5+7+1+1 = 17\text{ns}$

After pipeline cycle time: $3+5+1$ or $7+1+1 = 9\text{ns}$

Before pipeline Throughput: $1/17\text{ns} = 58.82\text{MOPS}$

After pipeline Throughput: $1/9\text{ns} = 111.11\text{MOPS}$

評分標準: Before 佔 1 分 , After 佔 1 分 , 若沒有單位扣 1 分

- (d) (3%) Will the latency be shortened after inserting the pipeline register as described in part (a)? Explain your reasoning. Assume that the cycle time is the shortest possible without causing a timing violation.

Latency 是輸入到輸出經過的時間 , 因此切 pipeline 不會讓 latency 減少

評分標準: 答案正確得 1 分 , 寫出原因得 2 分

- (e) (3%) Based on the pipeline you made in part (a), is it possible to further improve the throughput by adding another **one** pipeline register at one of the locations A, B, or C? Please explain why.

不行 , 因為切在 B 之後會有兩個 critical path , 把 register 擺在任一邊都無法使 cycle time 降低。

評分標準: 答案正確得 1 分 , 寫出原因得 2 分

8. (14%) Waveform.

Please refer to the following Pattern and Design code and draw the corresponding waveform diagram.

Please complete the waveform on the next page of this exam paper.

```

11 //-----
12 //  PATTERN
13 //-----
14 output logic clk, rst_n;
15 output logic in_valid;
16 output logic [3:0] in_num;
17 input out_valid;
18 input [3:0] out_num;
19
20 parameter CYCLE = 10;
21 always #(CYCLE/2.0) clk = ~clk;
22 initial clk = 0;
23
24 initial begin
25     in_valid = 0;
26     in_num = 0;
27     rst_n = 1;
28     @(negedge clk) rst_n = 0;
29     @(negedge clk) rst_n = 1;
30     in_valid = 1;
31     in_num = 8;
32     @(negedge clk);
33     in_num = 5;
34     @(negedge clk);
35     in_valid = 0;
36     in_num = 0;
37     wait(out_valid == 1);
38     repeat(2) @(negedge clk);
39     in_valid = 1;
40     in_num = 2;
41     @(negedge clk);
42     in_valid = 0;
43     in_num = 0;
44
45     repeat(10) @(negedge clk);
46     $finish;
47 end

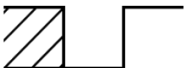
```

```

11 //-----
12 //  DESIGN
13 //-----
14 input clk, rst_n;
15 input in_valid;
16 input [3:0] in_num;
17 output logic out_valid;
18 output logic [3:0] out_num;
19
20 parameter IDLE=0, DIN=1, COM1=2, COM2=3, OUT=4;
21 logic [2:0] cs, ns;
22 logic [3:0] out_num_next;
23 logic signed [3:0] in_reg, in_reg_next;
24
25 always ff @(posedge clk) begin
26     if(!rst_n) cs <= IDLE;
27     else cs <= ns;
28 end
29 always_comb begin
30     case(cs)
31     IDLE:
32         if(in_valid) ns = DIN;
33         else ns = IDLE;
34     DIN:
35         if(!in_valid) ns = OUT;
36         else if(in_reg < 4) ns = COM1;
37         else ns = COM2;
38     COM1: ns = OUT;
39     COM2: ns = COM1;
40     OUT: ns = IDLE;
41     default: ns = 'dx;
42     endcase
43 end
44
45 always ff @(posedge clk) begin
46     if(!rst_n) in_reg <= 0;
47     else in_reg <= in_reg_next;
48 end
49 always_comb begin
50     if(in_valid) in_reg_next = in_num;
51     else in_reg_next = in_reg;
52 end
53
54 always ff @(posedge clk) begin
55     if(!rst_n) out_num <= 0;
56     else out_num <= out_num_next;
57 end
58 always_comb begin
59     if(cs == COM1) out_num_next = out_num + 2;
60     else if(cs == COM2) out_num_next = out_num >> 1;
61     else if(in_valid) out_num_next = in_num;
62     else out_num_next = in_reg;
63 end
64 assign out_valid = (cs == OUT)? 1: 0;

```

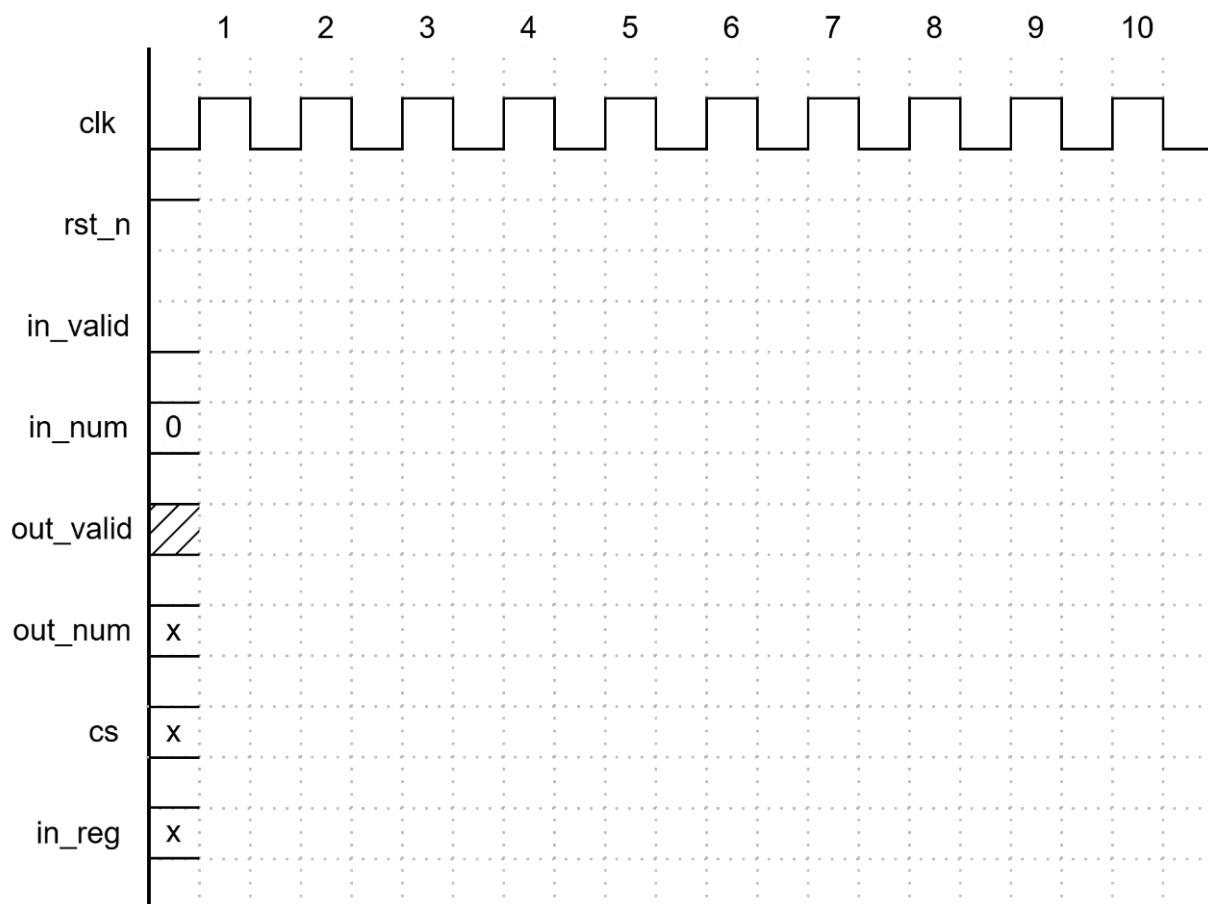
1-bit
signal



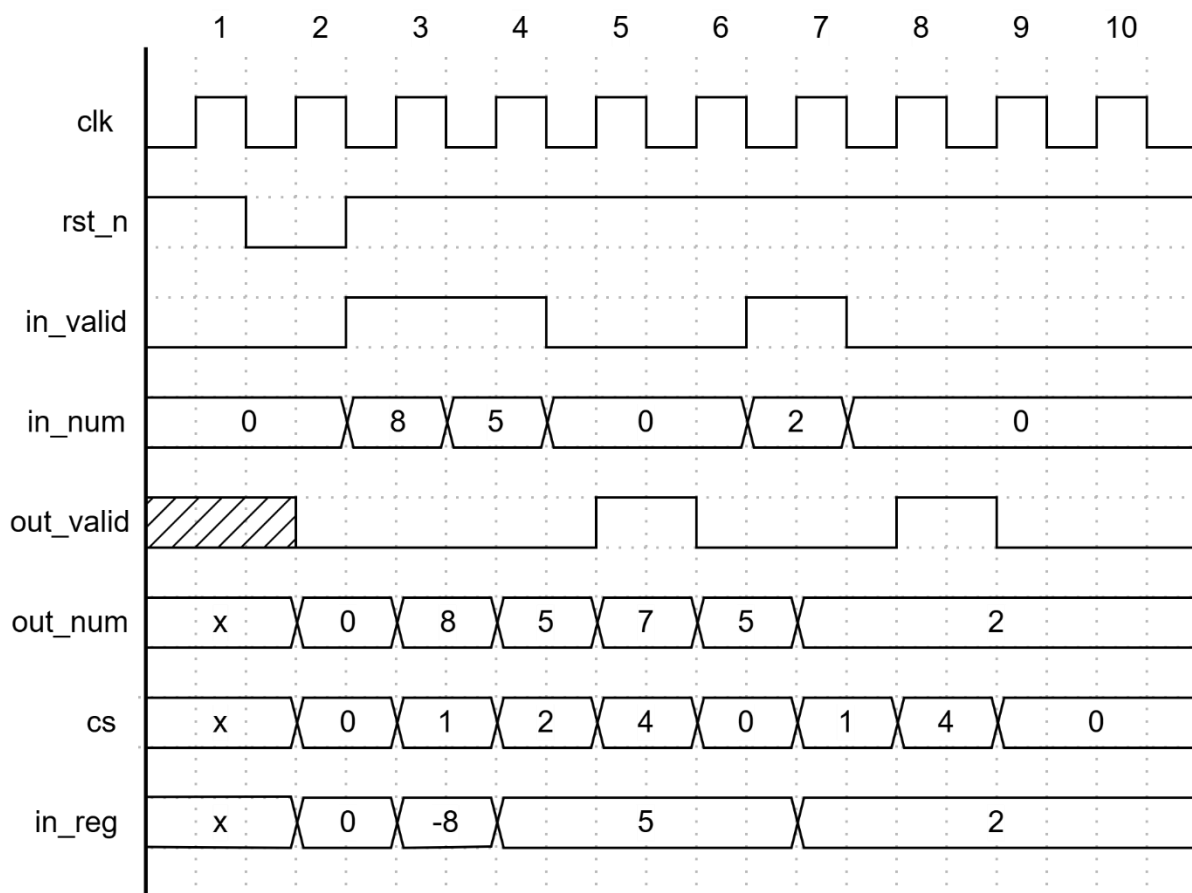
multi-bit
signal



The examples of drawing 1-bit and multi-bit signals are shown above. Please draw the waveforms for the signals `rst_n`, `in_valid`, `in_num`, `out_valid`, `out_num`, `cs`, and `in_reg` for at least 10 clock cycles. Note that the “`in_reg`” signal should be represented in signed 2’s complement format (e.g., mark -1 on waveform if `in_reg` = 4'b1111). All other signals are assumed to be unsigned.



Ans:



評分標準：

每個訊號 2 分，以下是常見扣分項目

Reset 錯 1 個訊號 → 扣 1 分

因為 in_reg = -8 導致 output 錯誤以及晚一個 cycle → 扣 4 分或 3 分

第二筆 input 沒有在 out_valid=1 後的下一個 negative edge 進來 → 扣 2 分

第二筆 input 的 cs 跳錯導致輸出錯誤 → 扣 3 分或 2 分

Pattern 給的 input 在 positive edge 變動 → 扣 4 分

其餘錯誤視情況扣分

9. (12%) For Flip-Flop A in Table 1 (used for DFF X, Y, Z), and a 2GHz clock, check the Fig. 1 (the symbol “ t_c ” represents the logic contamination delay, while “ t_d ” represents the logic propagation delay) for timing violations.

- (a) (4%) Is there any setup/hold time violation? If so, indicate where it happens along with the timing check formula.
- (b) (4%) How to solve it? And write the timing check formula after your modification.
- (c) (4%) Recheck for timing violations. Is it possible to run faster than 2GHz clock? Derive the cycle time for this maximum clock frequency?

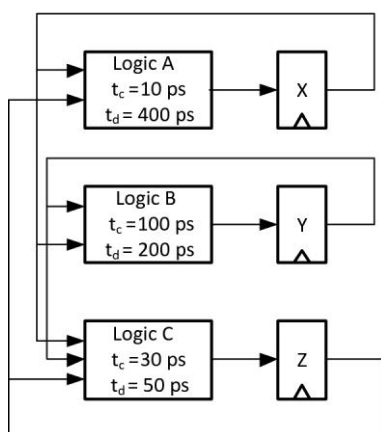


Fig. 1

Parameter (ps)	DFF A
Setup time: t_{setup}	30
Hold time: t_{hold}	35
Contamination delay, clock to q, t_{ccq}	10
Propagation delay, clock to q, t_{pcq}	20

Table 1

(Hint: Setup time check, $T_c \geq t_{\text{pcq}} + t_{\text{pd}} + t_{\text{setup}}$; Hold time check, $t_{\text{cd}} + t_{\text{ccq}} \geq t_{\text{hold}}$)

Ans:

(a) A 2GHz cycle time gives $T_c = 500\text{ps}$.

$$T_c \geq t_{\text{pcq}} + t_{\text{pdMax}} + t_{\text{setup}}$$

$500 \geq 20 + 400 + 30$, no setup time violation.

$$t_{\text{hold}} \leq t_{\text{ccq}} + t_{\text{cdMin}}$$

$$35 \leq 10 + 10$$

$35 \leq 20$ (violated), hold time violation at A→X.

(沒寫不等式 or 沒寫哪裡有 violation or 數字錯：扣兩分)

(b) add 15ps delay to logic A

$$t_{\text{hold}} \leq t_{\text{ccq}} + t_{\text{cdMin}}$$

$$35 \leq 10 + 25$$

(沒寫不等式 or 解法錯誤 or 數字錯：扣兩分)

(c) with extra 15ps delay, $T_c \geq 20 + 400 + 30 + 15 = 465 \text{ ps}$

(cycle time 數字錯：扣兩分)